Lexing and Grammars

CS/COE 1622

Jarrett Billingsley

Class Announcements

- project 1 is out if you didn't see the announcement(s)
  - remember it's due by **this Saturday evening**
  - or Sunday for late credit (-10%)
- finally today we're getting into the COMPILER STUFF!
- couple new examples in the examples repo for today too!

Lexing

What is it?

- when a compiler reads the source code, it's just a long **string.**
- **lexing** (or "scanning") is the process of splitting that string into small, meaningful pieces in order to simplify the next step, *parsing*.



the lexer's input is the **source code.**

the lexer's output is a list of **tokens.**

each token is like one "word" in the source language – the smallest unit of **meaning.**

Tokens, and… not-tokens

- **tokens** have meaning to the programming language. but **not everything** you type in the source code is meaningful.

```
if x < 10 {
  f();
}
```

```
if x<10{f();}
```

```
if x < 10
{
  // call
  f ( );
}
```

all three of these would lex to the **same sequence of tokens.** but what differs?

spacing and indentation – together called **whitespace** – is meaningless in *most* modern languages, and the lexer strips it out.

comments are another common kind of meaningless text that can be ignored.

Not-so-whitespace

- whitespace can't be ignored in *all* situations, though.

```
"hello, world"
"hello,    world"
```

these are different strings, right?

so we have to **remember** whether or not we're inside "quotes".

also, `fortress` and `for tress` mean different things.

Python uses **indentation** to structure code, instead of **{** braces **}.**

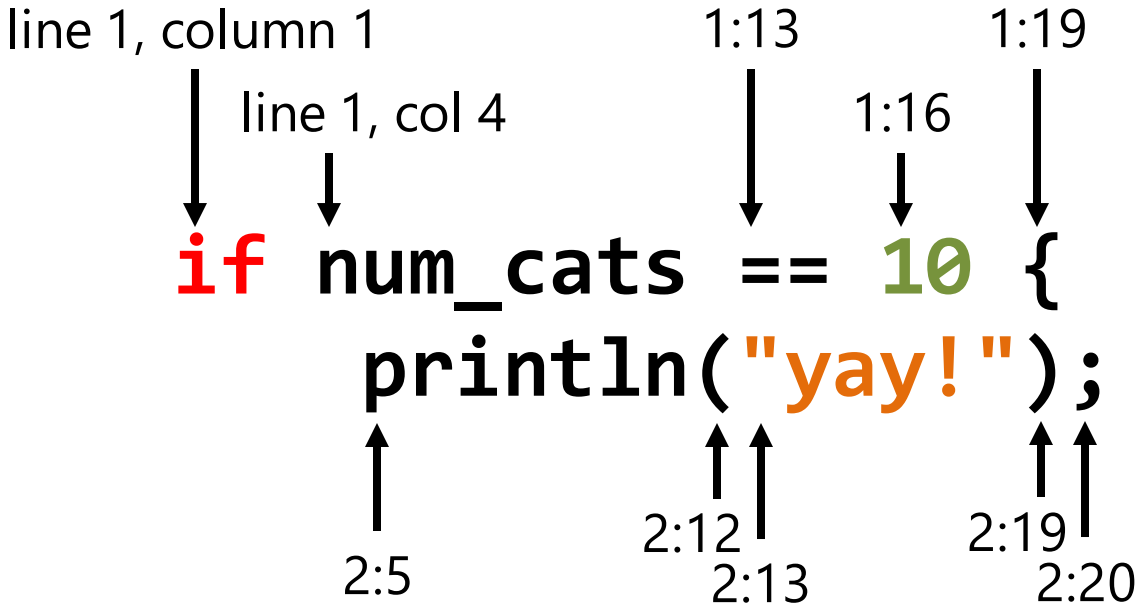
```
if x < 10:
    f()
print("done")
```

the **print** call is outside of the **if.** the only thing that indicates that is the indentation.

and **JavaScript** (aka ECMAScript) does *weird* things with newlines so you can avoid writing semicolons...

Where *are* we?

- a secondary (but *super important*) job of the lexer is to produce **location info** so the compiler can give good error messages.



the **line** number is based on how many newlines it's seen.

the **column** number is based on how many characters it's seen since the last newline.

this information can be carried forward through the rest of the compilation process for a number of uses.

Lexing from Intuition

Getting a feel for it

- a lot of problem-solving works by getting a feel for what a solution *might* look like, then **formalizing that intuition** into an algorithm.
- let's start by looking at what our input language looks like.

```
struct S {
  x: int,
  y: bool,
}

fn main() {
  println_s("hi!");
  let s = new S();
  s.x = 10;
  s.y = true;
  println_i(s.x);
}
```

this is the toy language we'll be using for projects and examples, called **Truss**.

it has similarities to both Rust and Java, but is a lot simpler in many ways.

what are some kinds of tokens you can pick out from this example?

Common classes of tokens

- *most* languages today use similar rules for their classes of tokens.

there are tokens that **look like words.**

```
struct  int
return  while
```

some have *special meaning* in the language. these are called **keywords.**

```
S   num_cats
println   main
```

others are written by the programmer to name things. these are called **identifiers.**

there are often many **symbols.**

```
+   -   *   /   =
.   ,   ;   {   }   ==
    &&  ||  +=
```

and there are **literals:** a way of embedding constant values directly into your code.

```
"hello,\nworld!"    'c'
true
        345     1.9e6   0xDEADBEEF
```

A first attempt

- we might come up with some simple rules for these tokens:
  - **identifiers** are a sequence of letters, underscores, and digits.
    - e.g. **`x`**, **`x2`**, **`_lift`**, **`THIS_IS_A_TEST`**, **`fort`**, **`o_o`**
  - **keywords** are a fixed subset of identifiers.
    - e.g. **`if`**, **`else`**, **`int`**, **`private`**, **`for`**
  - **symbols** are a fixed set of sequences of symbol characters.
    - e.g. **`+`**, **`+=`**, **`=`**, **`==`**, **`&`**, **`&=`**, **`&&`**
- but then we get to the literals, and it gets harder.
  - here are some floats: `1.2`, `10e9`, `6.28E+23`, `4.`, `5f`
  - do you think you could come up with a concise rule for that?
  - and then strings… `"hello\nworld"`
- oh, but it gets worse!

We keep running into each other

- let's look at some awkward situations!

`abc123` → `abc123`

`123abc` → `123abc`? *or* `123`, `abc`? *or* ***error?***

`x.y` → `x`, `.`, `y`

`4.y` → `4`, `.`, `y` *or* `4.`, `y` ?

`x.5` → `x`, `.`, `5` *or* `x`, `.5` ?

`4.5` → `4.5` *or* `4.`, `5` *or* `4`, `.`, `5` ?

`1.2.3` → `1.2`, `.`, `3`

or `1.2`, `.3`

or `1.`, `2.3`

or `1.`, `2.`, `3`

or `1`, `.`, `2`, `.`, `3`



uHH

- even with such a simple set of tokens, we're already running into problems, and it's not entirely clear where we went wrong.
- well, let's first learn a bit about **grammars,** which can give us some tools to talk about these things more rigorously.

Grammars

Languages and alphabets

- a **language** is a **set of strings.** (most *useful* languages are infinite sets.)
- each **string** is a sequence of symbols chosen from an **alphabet.**
- for example, here's a really dumb language and its alphabet:

```
L = { "hi", "bye" }
A = { 'a', 'b', …, 'z' }
```

each symbol in the alphabet is called a **terminal.**

if we generate strings from this alphabet, it's kind of obvious whether or not they are in L.

```
"hi"  ∈ L
"hii" ∉ L
"cat" ∉ L
```

…ok, what about a more complicated example?

Grammars

- for more complex or infinite languages, you need a **grammar:** a set of rules to decide if a string is in the language.

```
L = { "a", "aa", "aaa", … }
A = { 'a', 'b', …, 'z' }
```

what is/are the rule(s) for this language?

"1 or more 'a's"

okay. but can we write that grammar **rule** in a more *rigorous* way?

```
L : 'a'+
```

this is a **nonterminal.** it isn't part of the alphabet; it **stands in** for some combination of terminals.

this + says "repeat the previous thing **1 or more times.**"

A language for specifying languages

- we usually specify the lexical and syntactic form of a language with a **metalanguage:** a concise way of specifying rules.
- there are some common patterns for these rules:

`L: A B`	**A** followed by **B.** *(sequencing)*
`R: A | B`	an **A** *or* a **B.** *(alternation)*
`X: A*`	zero or more **A**s. *(repetition)*
`Y: A+`	one or more **A**s. *(repetition… again)*
`U: A?`	zero or one **A**, or "an optional **A**".
`D: (A B)+`	parens can group things together.

in these examples, A and B can be **terminals** (from the alphabet) *or* **nonterminals** (names of other rules).

Lexing a programming language

- the **alphabet** is the character set that the source code is in.
  - in our case, that's **Unicode!**
- the **language** isn't the *entire* programming language...
  - instead, it's **the tokens we want to produce.**
- so, we'll write a rule for each kind of token.

```
Symbol: '+' | '-' | '{' | '}' | '=' | ('=' '=')
```

'quoted' things are **terminals,** picked from the **alphabet.**

```
Keyword: ('i' 'f') | ('e' 'l' 's' 'e') |...
```

that's ugly, but we can make our metalanguage look any way we like.

```
Keyword: "if" | "else" | "fn" | "return"
```

that's better!

Getting more complex

- rules can refer to other rules. like here:

```
Id:         IdStart IdCont*
IdStart:    Alphabetic | '_' | '$'
IdCont:     IdStart | Digit
Digit:      '0'|'1'|'2'|'3'|'4'|'5'|'6'|'7'|'8'|'9'
Alphabetic: (a whole bunch of characters)
```

so, an identifier starts with a letter or underscore or dollar sign; and that is followed by zero or more of those, plus digits. neat.

and we could keep going for all the tokens!

Literals!

- numeric and string literals can get pretty complicated, but let's keep them simple for now:

```
IntLit:  Digit+
StrLit:  '"' StrChar* '"'
StrChar: <any character except '"'>
```

integer literals are sequences of digits, and string literals are sequences of characters surrounded by double quotes. also it's fine to be a bit hand-wavey like in **`StrChar`**, as long as the intent is clear.

but: what if the input contains the number 100000000000000000000? isn't it too big...?

defining the valid ranges of numbers *within the metalanguage* isn't possible. so we could define that as an additional rule.

Uh oh, whitespace.

- whitespace isn't a token, but we do have to deal with it somehow.
- we can write rules for it, like a real token:

```
Whitespace:  ' ' | '\t' | '\n' | Comment
Comment:     "//" CommentChar* CommentEnd
CommentChar: <any character except '\n' or Eof>
CommentEnd:  ('\n' | Eof)non-consuming
```



woah what the heck is that

this is a **lookahead:** it checks if the next character is a newline or EOF, but **it does not make it part of the Comment.**

it feels a little kludgey, but sometimes lookaheads are just what you need.

just… **don't use them too much.**

Wrapping it up and putting a bow on top

- once we have a rule for each kind of token, we can do this:

```
Token:   Symbol | Keyword | Id | IntLit | StrLit
Program: (Whitespace? Token)* Whitespace? Eof
```

here, **Program** is a special rule: it's our **top-level** or **start** rule.

if we want to translate this grammar into code that lexes, it's where we start – it's the outermost loop.

but wait. if we spent all this time writing the lexical rules in a formalized, rigorous way...

couldn't we just have a program **compile the grammar *itself* into a lexer?**

COMPILER COMPILERS 

- **yes, this is A Thing.**
- there are *many* tools which will take some kind of metalanguage as the input, and **produce a lexer program as an output.**
  - (or a parser, but we haven't gotten to those yet)
- however...
  - most are tightly tied to the language whose code they output.
  - they're really complex, to be as flexible as possible.
  - ...but, it can be awkward to fit your grammars into their rules.
  - they can give really confusing/terrible error messages on invalid lexer input.
- and honestly, writing your own lexer is *not that complicated*.
  - and, yanno, it's a compilers course, so you have to do it. (:

Dealing with Ambiguity

Dealing with ambiguity

- **ambiguity** is when you have > 1 possible "correct" tokenizations.
- consider these snippets of Java.

```
if(x == 3)
```
is this `[==]` or `[=, =]`?

```
3.4
```
is this `[3.4]` or `[3., 4]`?

```
abc123
```
is this `[abc123]` or `[abc, 123]` or `[ab, c123]` or...?

```
123abc
```
why do we think *this* is an error when the above is not?

```
int y = x >> 3;
```
on this line, `>>` is a right shift...

```
List<List<String>> l;
```
but here, it's two right angle brackets?

there are three ways I can think of to resolve ambiguity.

Approach 1: Maximal Munch

- **maximal munch** is a strategy that says: **always try to make the biggest token possible,** going left-to-right.

`if(x == 3)` this *is* `[==]`.

`3.4` this *is* `[3.4]`.

`abc123` this *is* `[abc123]`.

`123abc` this is an error because we **start** lexing a number, and then hit a non-numerical character.

although, it doesn't *have* to be; we could say "3x" is another way of writing "3 * x", couldn't we?

```
int y = x >> 3;
List<List<String>> l;
```

but this still presents an issue…

Approach 2: Deal with it later!

- we can have the parser (the next step) **disambiguate** things that would be **impossible to detect** in the lexer.

lexers are usually specified as *regular languages*,
which **cannot detect arbitrarily nested brackets.**

```
List<List<String>> l;
```

the lexer cannot know that `>>` is a pair of closing brackets
without knowing that there are two unpaired open brackets.

so, we have the *parser* say "oh, I currently have two open brackets;
this `>>` must be a pair of closing brackets and not a right-shift."

but *why bother making things harder for ourselves?*

Approach 3: "Doctor, it hurts when I do this"

- we could just **define our rules so they're unambiguous.**

```
List!(List!(String)) l;
```

in the **D language,** templates (generics) use parens instead.

the parser then knows **!(** is a "template open paren".

```
l :: List List String
m :: List (List String)
```

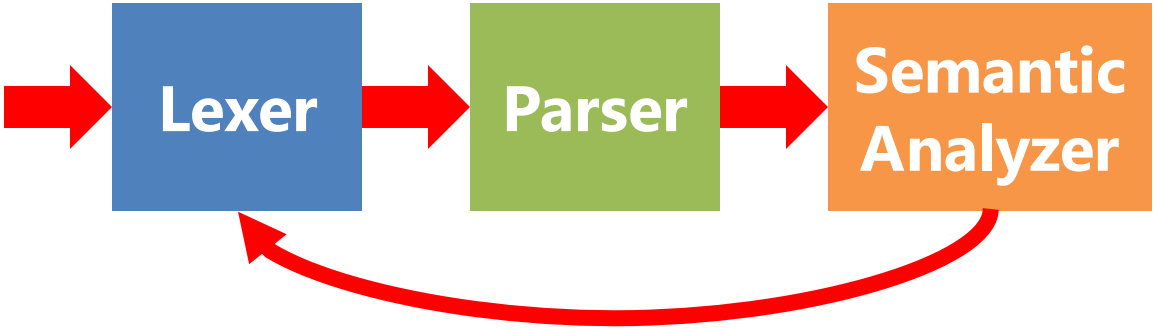
in **Haskell,** generics don't *require* any symbols, but *can* use parens.

```
x shr 3
```

or, we could redefine right shift to use some spelling other than >>.

Really bad cases of ambiguity

- if you aren't careful, you can end up with nasty situations.



C and C++ have this situation, where **you cannot correctly lex the source code until you've done (some) semantic analysis.**

this sucks! it makes the compiler slower, harder to write, and harder to reason about.

fortunately, we have the hindsight of decades of language design and can avoid this stuff :)

The examples

- first, there's an example showing how Rust's `match` statement works, called **rust_match.**
- there's a small example, **lexing_toy,** that shows how simple it can be to make a lexer.
  - it only has a few kinds of tokens, and is written in a different style from the project, cause I don't wanna give too much away (:
  - but it shows the general "shape" of a lexing algorithm.